Dynamic Merchandising Offer Block

1. Overview

The offer system delivers personalized promotional content via Adobe Target / AEP Edge Network, with fallback to default offer content when personalized offers are unavailable.

---

2. Current AEM Flow

2.1 Authoring & Setup (Pre-Runtime)

Step	Action	System
1	Marketing team creates and manages default offer content as Experience Fragments	Marketing AEM Author (Cloud)
2	Raw HTML of the default offer is copied from the publish instance	Marketing AEM Publish (Cloud)
3	An Experience Fragment is created with a Raw HTML component; the copied HTML is pasted into it	TFS AEM Author (On-Prem)
4	The Dynamic Offer Component is authored on the page, configured with a **Placement Code** (e.g., `mk_fp_01`) and a **Default Snippet Path** pointing to the Experience Fragment	TFS AEM Author (On-Prem)

2.2 Runtime Behavior

1. User accesses the page
2. Page HTML is served with offer placement containers (e.g., `<div id="tf-cq-mk_fp_01">`)
3. Adobe Launch loads and initializes the Alloy SDK (AEP Web SDK)
4. Alloy makes a `POST` call to `edge.adobedc.net/ee/v1/interact`
5. AEP Edge routes the request to Adobe Target for offer decisioning
6. **If Target returns an offer**: Alloy SDK processes the `setHtml` action and injects the Experience Fragment HTML into the matching container

7. **If Target returns no offer**: Default Offer is displayed.

2.3 Key Technical Findings

Experience Fragment HTML is clean and lightweight:

- The injected Experience Fragment content contains **ZERO scripts, ZERO stylesheets, ZERO iframes**
- It is composed entirely of divs, images, links, text, and hidden inputs
- This content is **not problematic for EDS pages** and can be safely injected at runtime

Offer rendering chain:

Impact on Lighthouse Score: The offer HTML itself is lightweight and will not degrade performance. Any Lighthouse impact will primarily be from **offer images** loaded from `dm-images.thermofisher.com` (Dynamic Media).

---

3. Assumptions

Assumption
Default offers are **not region/language specific** — stored under a common path (currently `/content/experience-fragments/tfsite/us/en/site/dm-offers/`). In EDS, default offer fragments will follow a similar centralized structure.
`dm-offers/preload.min.js` and `dm-offer-style.css` will be loaded from the global header/footer or via `delayed.js`. These are responsible for offer styling and fallback behavior.
The Adobe cloud pipeline (Marketing AEM → Target → AEP Edge → Alloy SDK → DOM injection) remains **unchanged**. Only the page-side implementation (how default content is stored and how placement containers are rendered) changes for EDS.
Placement codes are a known, finite set managed centrally (e.g., `mk_fp_01`, `mk_fp_02`, `hb`, `lb`, `pc`, `fad_s_1`).

---

4. EDS Implementation

Option A

Recommended way to implement in EDS would be to create Remote Offers in TFS instance. This means we simply reference the public URL (of a EDS fragment) from within Target.
Personalization will still be applied client side.

Option B

Unlike today where default offer HTML will be stored in fragments but default offer HTML may contain inline style,css, div's .

The EDS offer system consists of two components:

```
1. EMBED BLOCK (inside fragment)
   Purpose: Store raw HTML of default offer
   Location: /content/fragments/default-offers/

2. OFFER BLOCK (on the page)
   Purpose: Smart container for offer placement
   Handles: Target container, layout, fallback

```

4.2 Default Offer Fragment (Embed Block)

For each default offer, a **fragment page** is created containing an **embed block** that holds the raw HTML from Marketing AEM.

The default offer HTML from Marketing AEM contains inline styles, specific CSS classes, nested divs, hidden inputs, and data attributes. This raw HTML **cannot** be authored as standard EDS content — it must be stored and rendered as-is. The embed block serves this purpose.

Fragment path structure:

```
/content/fragments/default-offers/
  ├── mk-fp-01       ← fragment with embed block (raw HTML for Feature Panel Left)
  ├── mk-fp-02       ← fragment with embed block (raw HTML for Feature Panel Right)
  ├── hb             ← fragment with embed block (raw HTML for Header Banner)
  ├── lb             ← fragment with embed block (raw HTML for Lightbox)
  └── pc             ← fragment with embed block (raw HTML for Promo Cards)

```

Authoring workflow:

1. Marketing team creates/updates offer in Marketing AEM Author (Cloud) — *unchanged*
2. Raw HTML is copied from Marketing AEM Publish — *unchanged*

3. Author opens the corresponding fragment page in Universal Editor (e.g., `/content/fragments/default-offers/mk-fp-01`)
4. Pastes the raw HTML into the embed block
5. Publishes the fragment

The raw HTML (including inline styles, CSS classes, data attributes) is preserved as-is and served via `.plain.html`. When the offer block fetches this fragment on fallback, the HTML is injected into the DOM exactly as stored — maintaining visual parity with Target-delivered offers styled by the same `dm-offer-style.css`.

4.3 Offer Block (On Page)

The Offer block is the smart container placed on content pages. Authors add one or more offer items — the layout is automatically determined by the number of items.

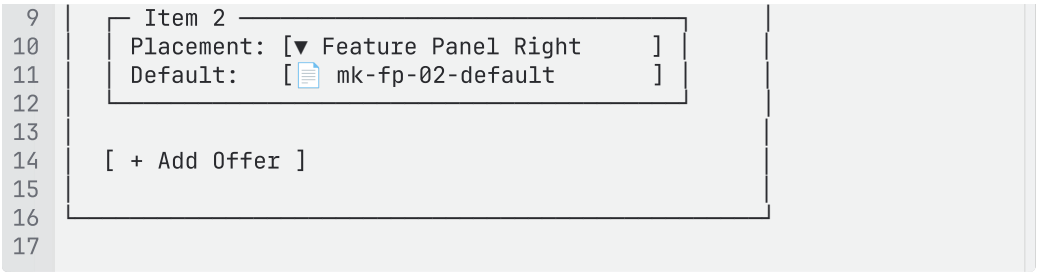
Offer block model (Universal Editor):

```
{
  "id": "offer",
  "fields": [
    {
      "component": "select",
      "name": "placementCode",
      "label": "Placement Code",
      "description": "Select the offer placement position",
      "valueType": "string",
      "options": [
        { "name": "Feature Panel Left", "value": "mk_fp_01" },
        { "name": "Feature Panel Right", "value": "mk_fp_02" },
        { "name": "Header Banner", "value": "hb" },
        { "name": "Lightbox", "value": "lb" },
        { "name": "Promo Cards", "value": "pc" },
        { "name": "Find A Distributor", "value": "fad_s_1" }
      ]
    },
    {
      "component": "aem-content",
      "name": "defaultOffer",
      "label": "Default Offer Fragment",
      "description": "Select the fragment containing fallback offer
HTML"
    }
  ]
}

```

Authoring in Universal Editor:

```
┌──────────────────────────────────────────┐
│  Offer Block Properties                  │
│                                          │
│   ┌─ Item 1 ──────────────────────────┐  │
│   │ Placement: [▼ Feature Panel Left  ] │  │
│   │ Default:   [📄 mk-fp-01-default   ] │  │
│   └──────────────────────────────────┘  │
│                                          │
```



No layout container or column configuration needed — the block auto-detects layout from item count.

4.4 Auto Layout — Dynamic Grid Based on Offer Count

Authors do not configure columns. The block counts the number of authored items and applies the matching grid layout automatically:

Author Action	Items	Resulting Layout
Add 1 offer item	1	Full width
Add 2 offer items	2	2-column grid
Add 3 offer items	3	3-column grid
Add 4 offer items	4	4-column grid

All four layout variations are **pre-defined in CSS**. The JavaScript dynamically applies the matching CSS class based on item count. No inline styles are generated at runtime.

4.5 Placement Code Dropdown — Options

Two options are available for populating the Placement Code dropdown in Universal Editor:

Option A: Static Options in Model

Placement codes are listed directly in the `component-models.json`:

```
"options": [
  { "name": "Feature Panel Left", "value": "mk_fp_01" },
  { "name": "Feature Panel Right", "value": "mk_fp_02" },
  { "name": "Header Banner", "value": "hb" },
  { "name": "Lightbox", "value": "lb" },
  { "name": "Promo Cards", "value": "pc" },
  { "name": "Find A Distributor", "value": "fad_s_1" }
]
```

Pros	Cons

Zero extra work, works out of the box	Developer must update `component-models.json` when new codes are added
No external dependencies	Requires code deployment for changes
Reliable and fast	

Option B: Dynamic via Spreadsheet + UE Field Extension

A spreadsheet (`placement-codes`) serves as the data source. A Universal Editor field extension fetches the spreadsheet JSON and populates the dropdown dynamically.

Spreadsheet: `placement-codes`

value	text	description
mk_fp_01	Feature Panel Left	Left feature panel on content pages
mk_fp_02	Feature Panel Right	Right feature panel on content pages
hb	Header Banner	Full-width header banner
lb	Lightbox	Lightbox popup offer
pc	Promo Cards	Promotional cards section

Pros	Cons
Marketing team manages codes via spreadsheet	Requires building a UE field extension
No developer needed for updates	Adds an external dependency
Self-service for marketing operations	

Recommendation: Start with Option A . Migrate to Option B only if placement codes change frequently and self-service management is a priority.

---

5. Open Questions

1. **What additional page-level data is required on EDS pages for Target decisioning?**
2. **Is `offers.min.js` still required in EDS**?
3. **Should default offer fallback be managed by `dm-offers/preload.min.js` or the EDS offer block?**
4. Confirm how currently default offer is shown is it managed by preload.js?
5. Confirm from PDP team that dynamic offers shown on PDP pages are coming from AEM or from AEP/Target

---